L.EIC: Functional and Logic Programming
First Project 2026/27

Pedro Vasconcelos, DCC/FCUP
Jácome Cunha, DEI/FEUP

October 14, 2026

1 Overview

1. This first project for 2026/2027 edition of the Functional and Logic Programming course consists of implementing a logical calculator.

2. This problem builds on the examples presented in Lectures 10 and 11 and some exercises from Worksheet 5. Make sure you have studied this background material before attempting to solve the project.

3. You should re-use the `Parsing.hs` module from Lecture 11 (available on the Moodle course page) unchanged and implemement a new main module as described below.

4. The project should be submitted individually on the public Codex server (see the Moodle course page) **before Friday 23:00 13th November 2026**.

5. Codex will test your submission using the example file below. Note that we will run extra test cases, so getting *Accepted* does **not** guarantee that you get full marks on the project. You should think of more test cases yourself.

6. You can discuss the problem with other students and professors. You can ask for help, or examine other students code as a means to understand the problem and its solution. However, you should attempt to then solve the problem yourself and not just submit someone else's solution.

7. There will be questions on the individual assements that are related the projects. Hence, you will be better prepared for the assessments if you use the projects as an oportunity for learning.

2 Description

Recall the data type for logical propositions presented in Lecture 10.

```
data Prop = Const Bool      -- constants (True/False)
          | Var Char        -- variables
          | And Prop Prop   -- conjunction
          | Or Prop Prop    -- disjuntion
          | Not Prop        -- negation
```

2.1 Task 1

Based on the parser for arithmetic expressions presented in Lecture 11, write a parser for `Prop`. Use the symbols '`|`' for disjunction, '`&`' for conjunction, '`!`' for negation and '`T`' and '`F`' for boolean constants. Variable names are single lowercase letters e.g. `x`, `y`, `p`, `q`. The complete grammar of propositions is as follows.

prop	::=	term propCont
propCont	::=	`|` term propCont | ε
term	::=	factor termCont
termCont	::=	`&` factor termCont | ε
factor	::=	`!` factor | name | constant | `(` prop `)`
constant	::=	`T` | `F`

Note that the grammar above gives higher precedence to conjunctions than disjunctions, e.g. `x|y&z` parses as `x|(y&z)`, *not* as `(x|y)&z`. Here are some example strings and the corresponding parsings into Haskell values:

Input string	Value
`a&b`	`And (Var 'a') (Var 'b')`
`a&b&c`	`And (And (Var 'a') (Var 'b')) (Var 'c')`
`a&b|c`	`Or (And (Var 'a') (Var 'b')) (Var 'c')`
`a&(b|c)`	`And (Var 'a') (Or (Var 'b') (Var 'c'))`
`!a|a`	`Ord (Not (Var 'a')) (Var 'a'))`
`T&!b`	`And (Const True) (Not (Var 'b'))`

2.2 Task 2

Using the calculator example of Lecture 11 as a basis, write a complete program that reads multiple lines from the standard input, parses each line as a proposition and outputs a suitable classification message:

`TAUTOLOGY` if the proposition is always true, for all possible truth value assignments of the variables;

`CONTRADICTION` if the proposition is always false, for all possible truth value assignments of the variables;

`CONTINGENT` if the proposition may be true or false, depending on the truth assignment of the variables.

Sugestion: Use the parser to convert each string into a `Prop` value, following the calculator example of Lecture 11. Write an auxiliary function

```
classify :: Prop → String
```

that classifies a single proposition. Use the functions of exercises 5.6–5.10 to compute the truth table for the proposition.

2.3 Example input/output

Sample input	**Sample output**
`T`	`TAUTOLOGY`
`a&F`	`CONTRADICTION`
`a`	`CONTINGENT`
`a|!a`	`TAUTOLOGY`
`!p&!!p`	`CONTRADICTION`
`p&T|p`	`CONTINGENT`
`p|!p&q`	`CONTINGENT`
`p&!p&p`	`CONTRADICTION`
`!(p&q)|p`	`TAUTOLOGY`